

EE6455 Fall 2025 - Gem5 Project

ResNet Workload Hardware Optimization

Due November 27, 2025

1 Overview

In this project you will design and optimize architecture configurations for a multi-threaded workload on ResNet network inference ([resnet_mt](#)). The architecture configuration includes cache hierarchy, coherence traffic, NoC topology, memory bandwidth, and CPU choice. Our goal is to optimize inference latency of the architecture by tuning parameters under a cost constraint.

2 Groups and Collaboration

- This assignment allows up to **2 students per group**.
- Each submission must include a **Division of Work Table** clearly stating each member's contribution. The following table is an example.

Team Member (Name / Student ID)	Tasks and Contributions	Effort (%)
Alex Chen / 114514000	Overall system integration, FS setup, CPU/memory tuning, report writing	50%
David Wang / 111062810	Cache hierarchy experiments, data collection, visualization and analysis	50%

3 Grading and Evaluation (100 pt total)

The final project grade is composed of three major components.

3.1 Baseline Correctness & Technical Report (70 pt)

- Your configuration must successfully execute under **Full-System Ruby/CHI** and produce valid output within the ResNet ROI.
- Proper ROI measurement, consistent `simSeconds` reporting, and reproducible scripts are required.
- The PDF report will be evaluated for:
 - Clarity and completeness of system setup and cost breakdown.

- Depth of analysis using quantitative evidence (`stats.txt`, plots, metrics).
- Logical explanation of design trade-offs and bottleneck analysis.
- Overall organization, figure labeling, and writing quality.

3.2 Verified Performance Ranking (20 pt)

- All groups will be ranked by their **TA-verified `simSeconds`**.
- The top-performing (lowest `simSeconds`) designs will earn proportionally higher scores.
- This portion rewards efficient architecture design under the fixed 1500-unit cost constraint.
- Any discrepancy exceeding $\pm 5\%$ will result in deduction from both ranking and report components.

3.3 Final Presentation (10 pt, top 10 groups only)

- The top 10 teams, determined by verified performance, will present their results in class.

4 Workload Description

The workload `resnet_mt.c` implements a compact ResNet-like inference model in C. Each Residual Block contains two 3×3 convolution layers with ReLU activations and a skip connection. Both convolution layers use zero padding to preserve the spatial resolution of feature maps. The model ends with a Global Average Pooling (GAP) layer followed by a Fully Connected (FC) layer that produces a softmax classification output, as illustrated in Fig. 1.

All data are single-precision (`float32`). Deterministic initialization ensures identical results across runs.

4.1 Workload Parameters

- `--channels (C)` : Number of feature channels
- `--size (H)` : Input height/width
- `--blocks (B)` : Number of residual blocks
- `--threads (T)` : Number of software threads (mapped to CPU cores)

4.2 Thread-to-Core Mapping and Workload Distribution

The program employs a lightweight thread pool implemented with POSIX `pthread` to achieve parallelism. Each worker thread is explicitly pinned to a specific CPU core using the POSIX function `pthread_setaffinity_np()`. This ensures consistent NUMA (Non-Uniform Memory Access) locality and prevents unwanted thread migration.

This deterministic thread-to-core binding makes timing behavior **reproducible and stable** in Full-System (FS) simulation.

At runtime, each thread reports its assigned physical core:

```
[Thread i] running on CPU <core> (target=i)
```

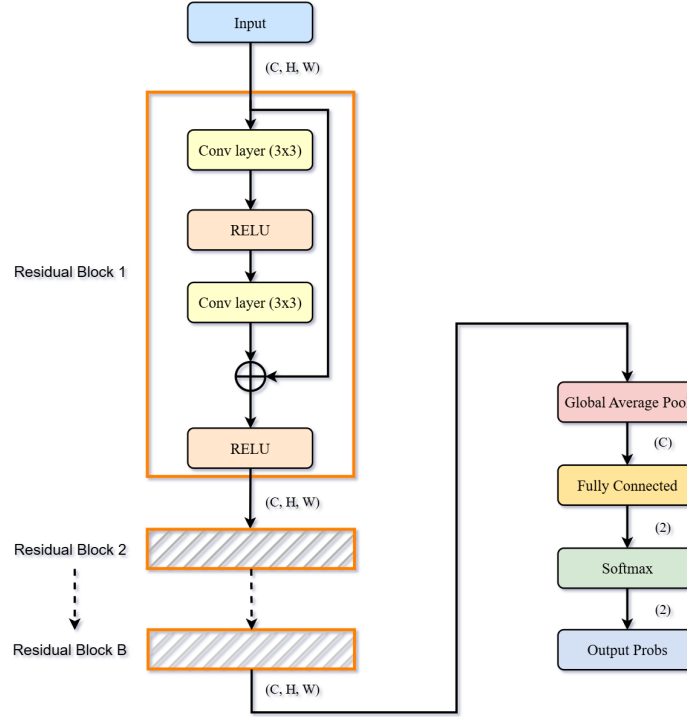


Figure 1: Structure of the compact ResNet-like model used in `resnet_mt.c`.

This message confirms the expected mapping between software threads and hardware cores, which is critical for analyzing **cache-sharing behavior** and **coherence traffic** under the Ruby/CHI hierarchy.

5 Hardware Constraint Parameters

5.1 Workload Configuration (Fixed for Fairness)

All groups must run the same workload configuration to ensure fairness:

```
--channels 16 --size 8 --blocks 4 --threads <N>
```

You can freely set the number of threads `<N>` to match number of cores in the configuration. Please **do not** modify the workload or source code.

5.2 Constraints and Design Rules

- **Full-System (FS)** mode with the **Ruby/CHI** coherence protocol. Classic coherence models are not accepted.
- You must employ **KVM fast-boot** for Linux startup and switch to a detailed timing CPU 03CPU in ROI simulation before collecting statistics.
- The total hardware cost of your configuration must not exceed **1500 units**.
- Only the parameters defined in the following Table 1 can be modified for optimization. All other parameters should remain fixed (Sec 5.4).

- All performance statistics must be recorded during the ResNet ROI execution phase.

5.3 Cost Table

Table 1 lists the configurable parameters for available components. Please use the table to calculate the total cost of your design.

- **Parameter Ranges (for O3 CPU):** To ensure fairness and prevent unrealistic scaling, all O3-related structural parameters must stay within the following ranges:
 - Fetch / Decode / Rename / Issue / Writeback / Commit widths ($W_f, W_d, W_r, W_i, W_w, W_c$): $1 \leq W \leq 32$
 - Reorder Buffer (ROB) entries (R): $16 \leq R \leq 512$
 - Load Queue (LQ) / Store Queue (SQ) entries: $16 \leq LQ, SQ \leq 512$
 - Physical Integer Registers (I): $32 \leq I \leq 512$
 - Physical Floating-Point Registers (F): $32 \leq F \leq 512$
 - Number of cores (N_{cores}): $1 \leq N_{\text{cores}} \leq 16$
- **Cache and Memory Constraints:**
 - L1/L2/L3 cache capacities must be realistic multiples of their cost base unit (e.g., 16 KB, 64 KB).
 - Memory channels: $1 \leq \text{channels} \leq 4$
 - Memory frequency: limited to available DDR3/DDR4 options (1600–3200 MHz).
- **Cost Rounding Rule:** The final total cost must be rounded to the nearest integer before submission. We only consider the integer value of the cost.

5.4 Fixed Parameters

The following parameters are fixed in the configuration.

Category	Parameter	Fixed Setting / Description
Simulation Mode	Full-System	Linux boot with Ruby/CHI
ISA	isa	X86
CPU Frequency	clk_freq	3 GHz
Boot Configuration	Fast boot method	Must use KVM CPU for Linux boot stage, then switch to O3 before entering ROI.
Software	resnet_mt binary	Pre-provided
OS disk Image	parsec.img	Pre-provided
OS kernel	vmlinux-4.4.186	Pre-provided

5.5 Design Constraints and Recommendations

- All cores must share the same frequency and cache coherence domain.
- CPU frequency is fixed at 3 GHz in this project (no frequency scaling).
- Cache topology (private/shared levels) must remain consistent with Ruby/CHI.
- The simulation must begin and end within the ResNet ROI.

Table 1: Cost table of configurable parameters

Component	Parameter	Cost Formula / Unit
CPU Core	O3 per-core base	40 per core
	O3 per-core structural score	$\text{score} = \alpha \overline{W} + \beta (R/16) + \gamma (I/32) + \gamma (F/32) + \delta ((LQ+SQ)/64) + \epsilon (W_i+W_c)$ where $\overline{W} = (W_f+W_d+W_r+W_i+W_w+W_c)/6$, W_f = fetch width, W_d = decode width, W_r = rename width, W_i = issue width, W_w = writeback width, W_c = commit width, R=ROB entries, I=phys. int regs, F=phys. FP regs, LQ, SQ=load/store queue entries. $(\alpha, \beta, \gamma, \delta, \epsilon) = (10, 12, 8, 6, 2)$
	Branch predictor option	$C_{BP} = 0$ (TournamentBP) $+5$ (TAGE/Perceptron)
	LQ/SQ entries (O3 only)	$C_{LSQ} = 0.05 \times \max(0, (LQ - 32) + (SQ - 32))$
	O3 per-core score cost	$C_{O3, \text{per-core}} = 40 + \kappa \cdot \text{score} + C_{BP} + C_{LSQ}$ where $\kappa = 0.01$ (fixed)
	Total O3 CPU cost	$C_{CPU} = N_{\text{cores}} \times C_{O3, \text{per-core}}$
Cache Hierarchy	L1 Instruction cache	3 per 16 KB
	L1 Data cache	3 per 16 KB
	L2 cache	1 per 16 KB
	L3 cache	1 per 64 KB
Memory System	Memory channel	60 per channel
	DRAM capacity	10 per GB
	Memory frequency	+5 per 400 MHz increase above 1600 MHz (increments applied in integer steps, up to 3200 MHz)
	DDR3 total cost	$C_{\text{mem3}} = (60 \times N_{\text{ch}}) + \left(5 \times \max\left(0, \left\lfloor \frac{f - 1600}{400} \right\rfloor\right) \times N_{\text{ch}} \right) + (10 \times \text{capacity}_{\text{GB}})$
	DDR4 adjustment	$C_{\text{mem4}} = 1.3 \times C_{\text{mem3}}$
Network Topology	Router node	10 per router node
	Interconnect link	5 per link between connected nodes
	Ring topology cost	$C_{\text{NoC}} = 10N_{\text{routers}} + 5N_{\text{links}}, N_{\text{links}} = N_{\text{routers}}$
	Pt2Pt topology cost	$C_{\text{NoC}} = 10N_{\text{routers}} + 5 \times \frac{N_{\text{routers}}(N_{\text{routers}}-1)}{2}$
	Router counting rule	$N_{\text{routers}} = N_{\text{L1I}} + N_{\text{L1D}} + N_{\text{L2}} + N_{\text{mem}} + N_{\text{DMA}}$ (+ N_{dir} if separate)
	Example interpretation	Each L1-I, L1-D, L2 slice/home, memory controller, or DMA Requestor (RubySequencer) counts as one router node; links are point-to-point connections between them.

6 Submission

Submit your work to eeclass as a single compressed archive named:

`gem5_project_groupNumber.zip`

Only one student from each team need to submit the file.

6.1 Archive Contents

Your submission must include the following directories and files:

- **scripts/** — All Python configuration scripts used in your design, including custom hierarchy, topology, and runner scripts (e.g., `hardware/fs_ruby_budget.py`).
- **results/** — Simulation logs and output files such as `stats.txt` for both baseline and optimized configurations, showing `simSeconds` measured strictly within the ROI.
- **report.pdf** — A concise technical report containing:
 - **Methodology:** Full-System (FS) setup, configuration parameters, and ROI measurement procedure.
 - **Design Description:** Hardware configuration, cost table (≤ 1500), and justification of each parameter choice.
 - **Performance Results:** Table summarizing `simSeconds` and total cost for all tested configurations, plus at least one plot (e.g., `SimSeconds vs Cost`).
 - **Analysis and Discussion:** Comparison against the baseline design, analysis of trade-offs, and bottleneck identification.
 - **Division of Work:** Clear summary of each team member's contribution and effort percentage.

Hint: Ensure that you start and stop simulation time measurement precisely within the ROI phase of the workload. Balancing cost investment across cores, caches, and memory bandwidth typically yields better overall performance. A moderately wide O3 CPU with sufficient DRAM throughput and a balanced Ring topology often outperforms a high-end CPU design bottlenecked by coherence or interconnect latency.

7 Final Presentation

After all submissions are verified, the **top 10 groups** — determined by the lowest TA-verified `simSeconds` — will be selected to deliver an in-class presentation.

Selected groups will present during class on December 8th (Mon). The detailed schedule and presentation order will be announced after all submissions are verified.

Each group must prepare a **(10-minute)** presentation with slides. The slides should include your design methodology, key experimental results, plots, and tables to visually support your analysis.

Tip: Focus on showing how your design achieved low `simSeconds` efficiently within the cost limit, supported by clear data and visual comparisons.

A Appendix A — Workload Distribution and Layer-wise Parallelism

A.1 Workload Distribution Strategy

The workload adopts **channel-wise parallelism** for compute-heavy operations such as convolution, where the work is primarily divided along the **output-channel (K)** dimension. Each thread is assigned a contiguous range of output channels — for example, with 16 output channels and 4 threads, each thread handles 4 channels. For other element-wise operations (e.g., ReLU, Add), the workload is distributed across the output tensor elements in a similar independent manner. This design ensures that threads operate on disjoint data regions, avoiding write conflicts and minimizing synchronization overhead.

A.2 Layer-wise Parallelism

Layer	Parallel Dimension	Thread Partition Strategy	Synchronization
Conv3x3	Output Channel (K)	Each thread computes a subset of output channels [ks, ke)	After full convolution
ReLU	Element Index ($N = C \times H \times W$)	Each thread processes a linear range of activations [s, e)	After activation
Add	Element Index ($N = C \times H \times W$)	Each thread performs $a[i] += b[i]$ over a subrange	After addition
GAP	Channel (C)	Each thread averages one or more channels independently	After pooling
Linear	Output Neuron (Out)	Each thread computes one or more output neurons	After linear layer

Synchronization occurs only **between layers** to ensure data consistency. This partitioning strategy maintains simple and predictable memory access patterns while avoiding false sharing, which is crucial for accurately studying **cache coherence traffic** and **memory bandwidth utilization** under the Ruby/CHI hierarchy.

B Appendix B — Suggested Observation Metrics

This appendix provides a list of **recommended metrics and visualizations** that may be used when analyzing the performance, cache behavior, and network characteristics of your design. You are not required to include all of them in your report — select the ones most relevant to your optimization goal.

B.1 Overall Simulation and CPU Metrics

Category	Metric	Description
Performance	<code>simSeconds</code>	Total simulated execution time.
CPU Utilization	<code>coresX.ipc</code>	Per-core IPC (instructions per cycle).
Derived Metric	$\text{Speedup} = \frac{\text{simSeconds}(T=1)}{\text{simSeconds}(T)}$	Parallel speedup relative to single-thread run.

B.2 Cache and Memory Metrics

Level	Metric	Description
L1/L2/L3	m_demand_accesses, m_demand_misses	Cache demand access and miss counters for each cache level. These indicate how often the CPU references each level of the memory hierarchy and how many of those accesses result in misses.
Derived Metric	MPKI (Misses Per Kilo-Instruction)	Measures cache pressure independently of IPC. A higher MPKI usually reflects poor cache locality or increased working set size.
Latency (Ruby Transaction)	inTransLatHist.Load::mean, inTransLatHist.Store::mean, inTransLatHist.SendReadShared::mean	Average latency of Ruby transactions (load, store, shared-read). This represents Ruby protocol latency rather than raw cache hit latency and is mainly used for comparing latency trends across L1/L2/L3 levels.

B.3 Ruby/CHI Coherence Transactions

Ruby (CHI) uses different naming conventions from the classical GETS/GETX/INV/FWD, but they are functionally equivalent. The following table summarizes the correspondence between classical and CHI-specific counters.

Classical Name	Ruby/CHI Counter Name	Meaning
GETS	ReadShared, SnpSharedFwd	Shared read request (non-owner data fetch).
GETX	WriteUniqueFull_PoC_Alloc, ReadUnique	Exclusive access request for write permission.
INV	WriteEvictFull, CleanUniqueEvict	Invalidation or write-back eviction event.
FWD	SnpSharedFwd	Forwarded snoop or probe for shared state.

B.4 Ruby Network Statistics

Metric	Example Counters	Description
Message Counts	reqOut.m_msg_count, rspIn.m_msg_count, datIn.m_msg_count, datOut.m_msg_count	Counts of request, response, and data messages exchanged within the Ruby network. These indicate overall coherence traffic between cache and memory controllers.
Mean Transaction Latency	outTransLatHist.mean	Average latency of Ruby transactions, including network traversal and protocol delays. Represents communication delay between cache and memory nodes.
Network Efficiency Metrics	average_hops, link_utilization	Average hop distance and link utilization across the interconnect. Shorter average hops indicate better locality, while higher link utilization suggests heavier NoC congestion.

B.5 Derived / Composite Metrics

Metric	Formula	Purpose
L1/L2/L3 MPKI	$\frac{\text{misses}}{\text{instructions}/1000}$	Quantifies cache hierarchy pressure.
DRAM requests per KI	$\frac{\text{memory_reads}}{\text{instructions}/1000}$	Measures off-chip memory bandwidth demand.
Network packets per instruction	$\frac{\text{reqOut} + \text{rspIn} + \text{datOut} + \text{datIn}}{\text{instructions}}$	Indicates NoC load per instruction.

B.6 Suggested Plots and Tables

Below are common visualization ideas for analyzing performance, scalability, and network behavior. You may select any subset that best supports your analysis.

Suggested Plot / Table	Description
SimSeconds vs Threads	Shows total runtime scaling — lower is better; reflects overall performance.
Speedup vs Threads	Evaluates parallel scalability relative to single-thread performance.
Efficiency vs Threads	Highlights parallel utilization efficiency, computed as $\text{Speedup} / \text{Threads}$.
Coherence Message Volume vs Threads	Shows how coherence message counts (GETS, GETX, INV, FWD) change with thread count.
Average Network Latency vs Threads	Useful for comparing Ring vs Point-to-Point interconnect congestion.
Cache Miss Rates vs Threads	Indicates cache locality and capacity behavior as thread count increases.
Load/Store Latency vs Threads	Helps identify contention or coherence-induced stalls.
Average Hops / Link Utilization	Evaluates NoC efficiency; shorter hops imply better locality, higher link utilization signals congestion.

C Appendix C — Suggested Steps and Deliverables

Follow the steps below to complete the performance-constrained hardware design.

C.1 Step 1 — Design Your Configuration.

Choose the CPU type, cache hierarchy, memory channels, and interconnect topology to achieve the best performance under the **1500-unit budget limit**. Compute all component costs using the formulas in the Cost Table and round only the *final total* to the nearest integer. Here is an example of a TA-provided hierarchy (Pt2Pt topology): [hierarchy_ruby_3level.py](#)

C.2 Step 2 — Summarize the Cost Breakdown.

Create a concise table showing how each subsystem contributes to total cost. A worked example (using an 8-core O3 design) is shown below:

Component	Specification	Cost (units)
$8 \times \text{O3 cores } (W=10, R=40, I=50, F=50, LQ=SQ=128)$	$8 \times C_{\text{O3,core}}=51.79$	414.32
L1 per core (32 KiB each)	$8 \times \left(\frac{32}{16} + \frac{32}{16}\right) \times 3$	96
Shared L2 (2×2 MiB slices)	$2 \times (2048/16)$	256
DDR4–2400 8GB dual channels	$1.3 \times [(60+10) \times 2 + 160]$	390
Ring topology (20 routers, 20 links)	$20 \times (10 + 5)$	300
Total		1456 (rounded)

You may reduce cache size or adjust the interconnect type if needed to remain under the 1500-unit cap.

C.3 Step 3 — Run the Simulation.

Execute your configuration in **Full-System (FS) mode** with **Ruby/CHI coherence**. Record the `simSeconds` value from `stats.txt`, measured strictly within the ResNet ROI (Region of Interest). This number determines your verified performance.

C.4 Step 4 — Analyze and Discuss.

Compare your optimized configuration against a baseline and discuss:

- How each design decision (CPU, cache, interconnect) affects performance per cost.
- Which subsystem yields the highest benefit per additional unit cost.
- Whether the final system is compute-bound, memory-bound, or coherence-bound.

D Appendix D — Example Cost Descriptions

D.1 Per-core Cost Calculation

For clarity, we show the per-core computation step-by-step according to the official cost table. Given an O3 core configured with:

$$W_f = W_d = W_r = W_i = W_w = W_c = 10, R = 40, I = 50, F = 50, LQ = 128, SQ = 128,$$

and constants $(\alpha, \beta, \gamma, \delta, \epsilon) = (10, 12, 8, 6, 2)$, $\kappa = 0.01$, and a TournamentBP predictor ($C_{BP} = 0$).

$$\overline{W} = \frac{10 + 10 + 10 + 10 + 10 + 10}{6} = 10.0$$

$$\text{score} = 10 \cdot 10.0 + 12 \cdot \frac{40}{16} + 8 \cdot \frac{50}{32} + 8 \cdot \frac{50}{32} + 6 \cdot \frac{(128 + 128)}{64} + 2 \cdot (10 + 10) = 219.0$$

$$C_{\text{LSQ}} = 0.05 \times ((128 - 32) + (128 - 32)) = 0.05 \times 192 = 9.6$$

$$C_{\text{O3, per-core}} = 40 + \kappa \cdot \text{score} + C_{BP} + C_{\text{LSQ}} = 40 + 0.01 \times 219.0 + 0 + 9.6 = 51.79$$

Hence, the precise per-core cost is **51.79 units**. For 8 cores:

$$C_{\text{CPU, total}} = 8 \times 51.79 = 414.32$$

D.2 Example A — Ring Topology

Configuration:

- $8 \times \text{O3 cores (as above)}$
- 32 KiB L1 per core
- $2 \times 2 \text{ MiB shared L2 slices}$
- 2 DDR4–2400 8GB
- Ring topology with 20 routers (8 L1I + 8 L1D + 2 L2 + 2 memory)

Component costs:

$$\begin{aligned}C_{\text{CPU}} &= 414.32 \\C_{\text{L1}} &= 8 \times \left(\frac{32}{16} + \frac{32}{16}\right) \times 3 = 96 \\C_{\text{L2}} &= 2 \times ((2048/16)) = 256 \\C_{\text{Mem}} &= 1.3 \times [(60+10) \times 2 + 160] = 390 \\C_{\text{Routers}} &= 20 \times 10 = 200 \\C_{\text{Links}} &= 20 \times 5 = 100\end{aligned}$$

Total (before rounding):

$$414.32 + 96 + 256 + 390 + 200 + 100 = 1456.32$$

After rounding to the nearest integer (round-half-up rule):

Example A (Ring) — Total = 1456 units

D.3 Example B — Point-to-Point (Pt2Pt) Topology

Same CPU/L1/Sequencer/Memory as Example A, but with a centralized 4 MiB L2 and a fully connected Pt2Pt interconnect.

$$\begin{aligned}C_{\text{CPU}} &= 414.32 \\C_{\text{L1}} &= 96 \\C_{\text{L2 (4 MiB)}} &= 256 \\C_{\text{Mem}} &= 390 \\C_{\text{Routers}} &= 19 \times 10 = 190 \quad (8 \text{ L1I} + 8 \text{ L1D} + 1 \text{ L2} + 2 \text{ mem}) \\C_{\text{Links}} &= \frac{19 \times 18}{2} \times 5 = 171 \times 5 = 855\end{aligned}$$

Total (before rounding):

$$414.32 + 96 + 256 + 390 + 190 + 855 = 2201.32$$

Rounded result:

Example B (Pt2Pt) — Total = 2201 units
--

Comment: If rounding were applied per core or different coefficients were used, slight variations (± 1 – 2 units) would appear. For consistency, this document applies “sum-then-round” only once at the end.

D.4 Summary of Both Topologies

- **Example A (Ring):** 1456 units (≤ 1500 budget)
- **Example B (Pt2Pt):** 2201 units (over budget)

The **Ring** interconnect scales linearly with node count and is thus practical for moderate core counts (4–8 cores). In contrast, **Pt2Pt** provides ideal latency but incurs quadratic link cost, becoming quickly unfeasible in large-scale systems. Under the current cost model, designs using Ring topologies can typically remain below 1500 units, while Pt2Pt networks often exceed the budget unless the system size is reduced to only a few cores.

E Appendix E — Practical Hint for Simulation Efficiency

E.1 Parallel Execution Tips

Since **gem5** executes as a single process on the host, each simulation run uses only one CPU core for event scheduling. To efficiently explore multiple configurations and reduce total runtime, it is recommended to launch several **independent simulations in parallel**, each with a different configuration and output directory.

- Run multiple configurations **in parallel** using separate terminals or background jobs.
- Always specify a **unique output directory** (`--outdir`) for each run to prevent data overwriting.

```
# Example: launch parallel runs for different configurations
# (e.g., varying core counts or cache sizes)

# Terminal 1
gem5 --outdir=m5out/config_A hardware/fs_ruby_A.py

# Terminal 2
gem5 --outdir=m5out/config_B hardware/fs_ruby_B.py

# Terminal 3
gem5 --outdir=m5out/config_C hardware/fs_ruby_C.py
```